5. Enumerations, Structures and Classes

Task 1:

Code:

```
import Foundation
enum BookStatus {
 case available
 case borrowed(by: String, dueIns: Int)
 case reserved(for: String)
 var label: String {
 switch self {
 case .available:
 return "Available"
 case .borrowed(let name, let days):
 return "Borrowed by \(name), due in \(days) days"
 case .reserved(let name):
 return "Reserved for \(name)"
 } } }
struct Book {
 var title: String
 var status: BookStatus = .available
}
class Member {
 var name: String
 var borrowedBooks: [String]
 init(name: String, borrowedBooks: [String] = []) {
 self.name = name
 self.borrowedBooks = borrowedBooks
 } }
```

```
var book1 = Book(title: "Swift Basics")
var book2 = book1
book2.status = .borrowed(by: "Deepika", dueIns: 5)
print("STRUCT EXAMPLE")
print("Book 1: \(book1.title), \(book1.status.label)")
print("Book 2: \(book2.title), \(book2.status.label)")
let member1 = Member(name: "Deepika")
let member2 = member1
member2.name = "Brinda"
print("\nCLASS EXAMPLE")
print("Member 1: \(member1.name)")
print("Member 2: \(member2.name)")
```

OUTPUT:

Output

```
STRUCT EXAMPLE
Book 1: Swift Basics, Available
Book 2: Swift Basics, Borrowed by Deepika, due in 5 days

CLASS EXAMPLE
Member 1: Brinda
Member 2: Brinda
```

Task 2:

Code:

```
import Foundation
enum Shape {
    case circle(radius: Double)
    case rectangle(width: Double, height: Double)
    case triangle(base: Double, height: Double)
```

```
    var area: Double {
        switch self {
        case .circle(let radius):
            return Double.pi * radius * radius
        case .rectangle(let width, let height):
            return width * height
        case .triangle(let base, let height):
            return 0.5 * base * height
        } } }
struct Rectangle {
    var width: Double
    var height: Double
    var perimeter: Double {
        return 2 * (width + height)
    }
    mutating func scale(by factor: Double) {
        width *= factor
        height *= factor
    }
}
let circle = Shape.circle(radius: 5)
let shapeRectangle = Shape.rectangle(width: 4, height: 6)
let triangle = Shape.triangle(base: 8, height: 4)
print("SHAPE AREAS")
print("Circle area: \(circle.area)")
print("Rectangle area: \(shapeRectangle.area)")
print("Triangle area: \(triangle.area)")
var rectangle1 = Rectangle(width: 10, height: 5)
var rectangle2 = rectangle1
```

```
print("\nBEFORE SCALING")
print("Original: \(rectangle1.width) x \(rectangle1.height)")
print("Original perimeter: \(rectangle1.perimeter)")
rectangle2.scale(by: 2)
print("\nAFTER SCALING THE COPY")
print("Original: \(rectangle1.width) x \(rectangle1.height)")
print("Copy: \(rectangle2.width) x \(rectangle2.height)")
print("Original perimeter: \(rectangle1.perimeter)")
print("Copy perimeter: \(rectangle2.perimeter)")
```

OUTPUT:

```
Output

SHAPE AREAS
Circle area: 78.53981633974483
Rectangle area: 24.0
Triangle area: 16.0

BEFORE SCALING
Original: 10.0 x 5.0
Original perimeter: 30.0

AFTER SCALING THE COPY
Original: 10.0 x 5.0
Copy: 20.0 x 10.0
Original perimeter: 30.0
Copy perimeter: 60.0
```

Task 3:

Code:

```
struct Counter {
    var value = 0
    mutating func increment() {
        value += 1
    } }
class Box {
```

```
    var value = 0
    func increment() {
        value += 1
    } }
let c = Counter()
c.increment()
let b = Box()
b.increment()
```

OUTPUT:



Explanation:

The error occurs because `c` is declared using let, which means it is a constant. In Swift, a constant struct cannot call a mutating method because that method changes its value.

Error line:

```
let c = Counter()
c.increment()
```

Correction:

```
var c = Counter()
c.increment()
```

Correct Code:

```
import Foundation

struct Counter {

    var value = 0

    mutating func increment() {

        value += 1

    } }

class Box {

    var value = 0

    func increment() {

        value += 1

    } }

var c = Counter()

c.increment()

print("Counter value: \(c.value)")

let b = Box()

b.increment()

print("Box value: \(b.value)")

let b1 = b

let b2 = b

print("Same object: \(b1 === b2)")
```

OUTPUT:

Output

```
Counter value: 1
Box value: 1
Same object: true
```